

Chương 6 – Cây quyết định (Decision Trees)

Notebook này chứa toàn bộ mã nguồn ví dụ và lời giải cho các bài tập trong chương 6.

 [Open in Colab](#)  [Open in Kaggle](#)

✓ Thiết lập (Setup)

Dự án này yêu cầu Python 3.7 trở lên:

```
import sys

assert sys.version_info >= (3, 7)
```

Nó cũng yêu cầu Scikit-Learn $\geq 1.0.1$:

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

Như chúng ta đã làm ở các chương trước, hãy định nghĩa kích thước phông chữ mặc định để làm cho các hình vẽ đẹp hơn:

```
import matplotlib.pyplot as plt

plt.rc('font', size=14)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)
```

Và hãy tạo thư mục `images/decision_trees` (nếu nó chưa tồn tại), và định nghĩa hàm `save_fig()` được sử dụng trong notebook này để lưu các hình ảnh ở độ phân giải cao cho sách:

```
from pathlib import Path

IMAGES_PATH = Path() / "images" / "decision_trees"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

✓ Huấn luyện và Trực quan hóa Cây quyết định

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris(as_frame=True)
X_iris = iris.data[["petal length (cm)", "petal width (cm)"]].values
y_iris = iris.target

tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X_iris, y_iris)

DecisionTreeClassifier(max_depth=2, random_state=42)
```

Đoạn mã này tạo ra Hình 6–1. Cây quyết định Iris:

```

from sklearn.tree import export_graphviz

export_graphviz(
    tree_clf,
    out_file=str(IMGES_PATH / "iris_tree.dot"), # path differs in the book
    feature_names=["petal length (cm)", "petal width (cm)"],
    class_names=iris.target_names,
    rounded=True,
    filled=True
)

```

```

from graphviz import Source

Source.from_file(IMGES_PATH / "iris_tree.dot") # path differs in the book

```

Graphviz cũng cung cấp công cụ dòng lệnh `dot` để chuyển đổi các tệp `.dot` sang nhiều định dạng khác nhau. Lệnh sau đây chuyển đổi tệp dot thành hình ảnh png:

```

# extra code
!dot -Tpng {IMGES_PATH / "iris_tree.dot"} -o {IMGES_PATH / "iris_tree.png"}

```

✦ Đưa ra dự đoán (Making Predictions)

```

import numpy as np
import matplotlib.pyplot as plt

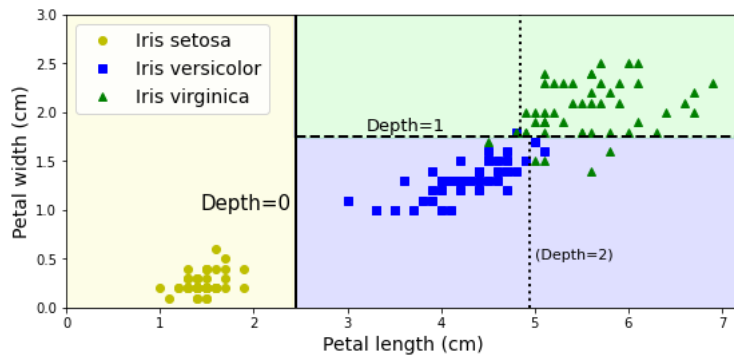
# extra code - just formatting details
from matplotlib.colors import ListedColormap
custom_cmap = ListedColormap(['#fafab0', '#9898ff', '#a0faa0'])
plt.figure(figsize=(8, 4))

lengths, widths = np.meshgrid(np.linspace(0, 7.2, 100), np.linspace(0, 3, 100))
X_iris_all = np.c_[lengths.ravel(), widths.ravel()]
y_pred = tree_clf.predict(X_iris_all).reshape(lengths.shape)
plt.contourf(lengths, widths, y_pred, alpha=0.3, cmap=custom_cmap)
for idx, (name, style) in enumerate(zip(iris.target_names, ("yo", "bs", "g^"))):
    plt.plot(X_iris[:, 0][y_iris == idx], X_iris[:, 1][y_iris == idx],
             style, label=f"Iris {name}")

# extra code - this section beautifies and saves Figure 6-2
tree_clf_deeper = DecisionTreeClassifier(max_depth=3, random_state=42)
tree_clf_deeper.fit(X_iris, y_iris)
th0, th1, th2a, th2b = tree_clf_deeper.tree_.threshold[[0, 2, 3, 6]]
plt.xlabel("Petal length (cm)")
plt.ylabel("Petal width (cm)")
plt.plot([th0, th0], [0, 3], "k-", linewidth=2)
plt.plot([th0, 7.2], [th1, th1], "k--", linewidth=2)
plt.plot([th2a, th2a], [0, th1], "k:", linewidth=2)
plt.plot([th2b, th2b], [th1, 3], "k:", linewidth=2)
plt.text(th0 - 0.05, 1.0, "Depth=0", horizontalalignment="right", fontsize=15)
plt.text(3.2, th1 + 0.02, "Depth=1", verticalalignment="bottom", fontsize=13)
plt.text(th2a + 0.05, 0.5, "(Depth=2)", fontsize=11)
plt.axis([0, 7.2, 0, 3])
plt.legend()
save_fig("decision_tree_decision_boundaries_plot")

plt.show()

```



Bạn có thể truy cập cấu trúc cây thông qua thuộc tính `tree_`:

```
tree_clf.tree_
```

```
<sklearn.tree._tree.Tree at 0x7fbfa8b563b0>
```

Để biết thêm thông tin, hãy kiểm tra tài liệu (documentation) của lớp này:

```
# help(sklearn.tree._tree.Tree)
```

See the extra material section below for an example.

✓ Ước lượng xác suất các lớp (Estimating Class Probabilities)

```
tree_clf.predict_proba([[5, 1.5]]).round(3)
```

```
array([[0.    , 0.907, 0.093]])
```

```
tree_clf.predict([[5, 1.5]])
```

```
array([1])
```

✓ Siêu tham số điều chuẩn (Regularization Hyperparameters)

```
from sklearn.datasets import make_moons
```

```
X_moons, y_moons = make_moons(n_samples=150, noise=0.2, random_state=42)
```

```
tree_clf1 = DecisionTreeClassifier(random_state=42)
```

```
tree_clf2 = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
```

```
tree_clf1.fit(X_moons, y_moons)
```

```
tree_clf2.fit(X_moons, y_moons)
```

```
DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
```

```
# extra code - this cell generates and saves Figure 6-3
```

```
def plot_decision_boundary(clf, X, y, axes, cmap):
```

```
    x1, x2 = np.meshgrid(np.linspace(axes[0], axes[1], 100),
                        np.linspace(axes[2], axes[3], 100))
```

```
    X_new = np.c_[x1.ravel(), x2.ravel()]
```

```
    y_pred = clf.predict(X_new).reshape(x1.shape)
```

```
    plt.contourf(x1, x2, y_pred, alpha=0.3, cmap=cmap)
```

```
    plt.contour(x1, x2, y_pred, cmap="Greys", alpha=0.8)
```

```
    colors = {"Wistia": ["#78785c", "#c47b27"], "Pastel1": ["red", "blue"]}
```

```
    markers = ("o", "^")
```

```
    for idx in (0, 1):
```

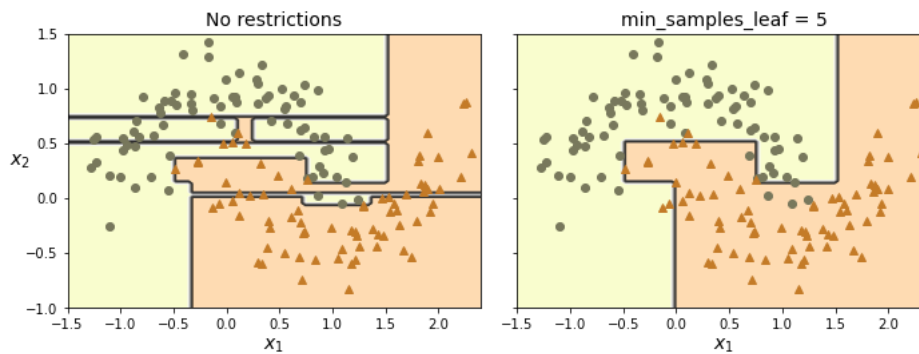
```
        plt.plot(X[:, 0][y == idx], X[:, 1][y == idx],
```

```
                color=colors[cmap][idx], marker=markers[idx], linestyle="none")
```

```
    plt.axis(axes)
```

```
plt.xlabel(r"$x_1$")
plt.ylabel(r"$x_2$", rotation=0)

fig, axes = plt.subplots(ncols=2, figsize=(10, 4), sharey=True)
plt.sca(axes[0])
plot_decision_boundary(tree_clf1, X_moons, y_moons,
                        axes=[-1.5, 2.4, -1, 1.5], cmap="Wistia")
plt.title("No restrictions")
plt.sca(axes[1])
plot_decision_boundary(tree_clf2, X_moons, y_moons,
                        axes=[-1.5, 2.4, -1, 1.5], cmap="Wistia")
plt.title(f"min_samples_leaf = {tree_clf2.min_samples_leaf}")
plt.ylabel("")
save_fig("min_samples_leaf_plot")
plt.show()
```



```
X_moons_test, y_moons_test = make_moons(n_samples=1000, noise=0.2,
                                          random_state=43)
tree_clf1.score(X_moons_test, y_moons_test)
```

0.898

```
tree_clf2.score(X_moons_test, y_moons_test)
```

0.92

▽ Hồi quy (Regression)

Hãy chuẩn bị một tập dữ liệu huấn luyện bậc hai đơn giản:

Code example:

```
from sklearn.tree import DecisionTreeRegressor

np.random.seed(42)
X_quad = np.random.rand(200, 1) - 0.5 # a single random input feature
y_quad = X_quad ** 2 + 0.025 * np.random.randn(200, 1)

tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X_quad, y_quad)
```

```
DecisionTreeRegressor(max_depth=2, random_state=42)
```

```
# extra code - we've already seen how to use export_graphviz()
export_graphviz(
    tree_reg,
    out_file=str(IMAGE_PATH / "regression_tree.dot"),
    feature_names=["x1"],
    rounded=True,
    filled=True
)
Source.from_file(IMAGE_PATH / "regression_tree.dot")
```

```
tree_reg2 = DecisionTreeRegressor(max_depth=3, random_state=42)
tree_reg2.fit(X_quad, y_quad)
```

```
DecisionTreeRegressor(max_depth=3, random_state=42)
```

```
tree_reg.tree_.threshold
```

```
array([-0.30265072, -0.40830374, -2.          , -2.          ,  0.27175756,
       -2.          , -2.          ])
```

```
tree_reg2.tree_.threshold
```

```
array([-0.30265072, -0.40830374, -0.45416115, -2.          , -2.          ,
       -0.37022041, -2.          , -2.          ,  0.27175756, -0.21270403,
       -2.          , -2.          ,  0.40399227, -2.          , -2.          ])
```

extra code - this cell generates and saves Figure 6-5

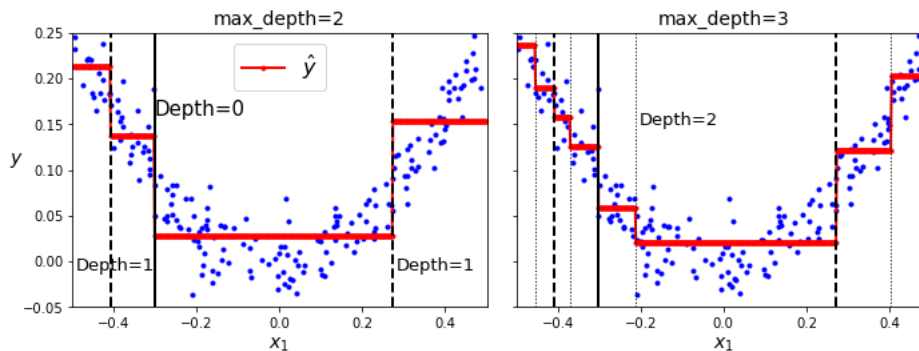
```
def plot_regression_predictions(tree_reg, X, y, axes=[-0.5, 0.5, -0.05, 0.25]):
    x1 = np.linspace(axes[0], axes[1], 500).reshape(-1, 1)
    y_pred = tree_reg.predict(x1)
    plt.axis(axes)
    plt.xlabel("$x_1$")
    plt.plot(X, y, "b.")
    plt.plot(x1, y_pred, "r.-", linewidth=2, label=r"$\hat{y}$")

fig, axes = plt.subplots(ncols=2, figsize=(10, 4), sharey=True)
plt.sca(axes[0])
plot_regression_predictions(tree_reg, X_quad, y_quad)

th0, th1a, th1b = tree_reg.tree_.threshold[[0, 1, 4]]
for split, style in ((th0, "k-"), (th1a, "k--"), (th1b, "k--")):
    plt.plot([split, split], [-0.05, 0.25], style, linewidth=2)
plt.text(th0, 0.16, "Depth=0", fontsize=15)
plt.text(th1a + 0.01, -0.01, "Depth=1", horizontalalignment="center", fontsize=13)
plt.text(th1b + 0.01, -0.01, "Depth=1", fontsize=13)
plt.ylabel("$y$", rotation=0)
plt.legend(loc="upper center", fontsize=16)
plt.title("max_depth=2")

plt.sca(axes[1])
th2s = tree_reg2.tree_.threshold[[2, 5, 9, 12]]
plot_regression_predictions(tree_reg2, X_quad, y_quad)
for split, style in ((th0, "k-"), (th1a, "k--"), (th1b, "k--")):
    plt.plot([split, split], [-0.05, 0.25], style, linewidth=2)
for split in th2s:
    plt.plot([split, split], [-0.05, 0.25], "k:", linewidth=1)
plt.text(th2s[2] + 0.01, 0.15, "Depth=2", fontsize=13)
plt.title("max_depth=3")

save_fig("tree_regression_plot")
plt.show()
```



extra code - this cell generates and saves Figure 6-6

```
tree_reg1 = DecisionTreeRegressor(random_state=42)
tree_reg2 = DecisionTreeRegressor(random_state=42, min_samples_leaf=10)
tree_reg1.fit(X_quad, y_quad)
tree_reg2.fit(X_quad, y_quad)
```

```

x1 = np.linspace(-0.5, 0.5, 500).reshape(-1, 1)
y_pred1 = tree_reg1.predict(x1)
y_pred2 = tree_reg2.predict(x1)

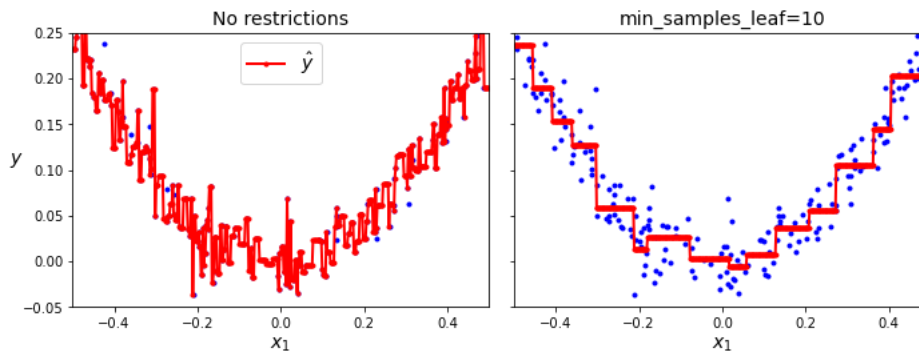
fig, axes = plt.subplots(ncols=2, figsize=(10, 4), sharey=True)

plt.sca(axes[0])
plt.plot(X_quad, y_quad, "b.")
plt.plot(x1, y_pred1, "r.-", linewidth=2, label=r"$\hat{y}$")
plt.axis([-0.5, 0.5, -0.05, 0.25])
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.legend(loc="upper center")
plt.title("No restrictions")

plt.sca(axes[1])
plt.plot(X_quad, y_quad, "b.")
plt.plot(x1, y_pred2, "r.-", linewidth=2, label=r"$\hat{y}$")
plt.axis([-0.5, 0.5, -0.05, 0.25])
plt.xlabel("$x_1$")
plt.title(f"min_samples_leaf={tree_reg2.min_samples_leaf}")

save_fig("tree_regression_regularization_plot")
plt.show()

```



▼ Độ nhạy đối với hướng của trục

Xoay tập dữ liệu cũng dẫn đến các ranh giới quyết định hoàn toàn khác nhau:

```

# extra code - this cell generates and saves Figure 6-7

np.random.seed(6)
X_square = np.random.rand(100, 2) - 0.5
y_square = (X_square[:, 0] > 0).astype(np.int64)

angle = np.pi / 4 # 45 degrees
rotation_matrix = np.array([[np.cos(angle), -np.sin(angle)],
                             [np.sin(angle), np.cos(angle)]])
X_rotated_square = X_square.dot(rotation_matrix)

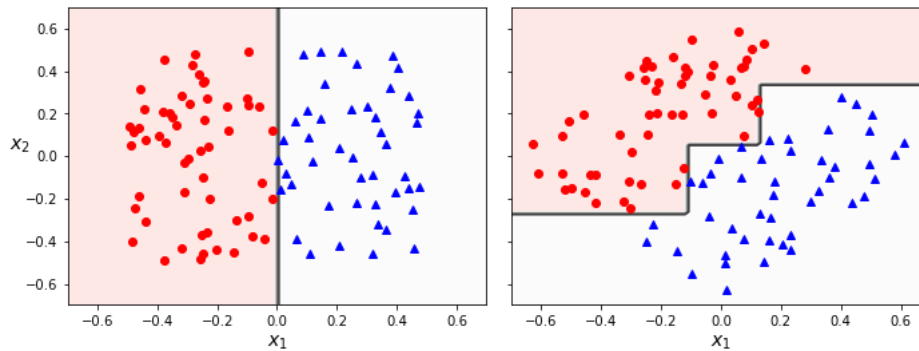
tree_clf_square = DecisionTreeClassifier(random_state=42)
tree_clf_square.fit(X_square, y_square)
tree_clf_rotated_square = DecisionTreeClassifier(random_state=42)
tree_clf_rotated_square.fit(X_rotated_square, y_square)

fig, axes = plt.subplots(ncols=2, figsize=(10, 4), sharey=True)
plt.sca(axes[0])
plot_decision_boundary(tree_clf_square, X_square, y_square,
                        axes=[-0.7, 0.7, -0.7, 0.7], cmap="Pastel1")

plt.sca(axes[1])
plot_decision_boundary(tree_clf_rotated_square, X_rotated_square, y_square,
                        axes=[-0.7, 0.7, -0.7, 0.7], cmap="Pastel1")
plt.ylabel("")

save_fig("sensitivity_to_rotation_plot")
plt.show()

```



```
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

pca_pipeline = make_pipeline(StandardScaler(), PCA())
X_iris_rotated = pca_pipeline.fit_transform(X_iris)
tree_clf_pca = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf_pca.fit(X_iris_rotated, y_iris)
```

```
DecisionTreeClassifier(max_depth=2, random_state=42)
```

extra code - this cell generates and saves Figure 6-8

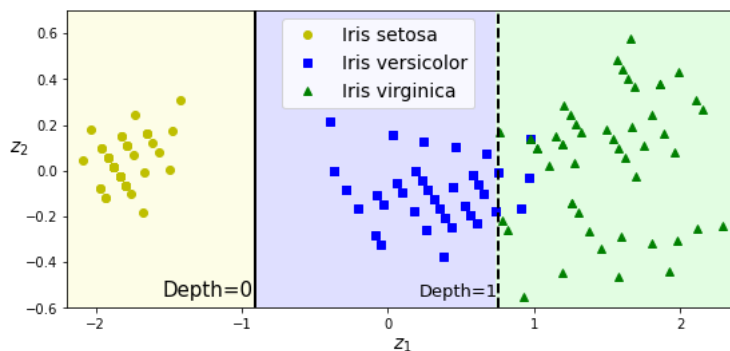
```
plt.figure(figsize=(8, 4))

axes = [-2.2, 2.4, -0.6, 0.7]
z0s, z1s = np.meshgrid(np.linspace(axes[0], axes[1], 100),
                        np.linspace(axes[2], axes[3], 100))
X_iris_pca_all = np.c_[z0s.ravel(), z1s.ravel()]
y_pred = tree_clf_pca.predict(X_iris_pca_all).reshape(z0s.shape)

plt.contourf(z0s, z1s, y_pred, alpha=0.3, cmap=custom_cmap)
for idx, (name, style) in enumerate(zip(iris.target_names, ("yo", "bs", "g^"))):
    plt.plot(X_iris_rotated[:, 0][y_iris == idx],
             X_iris_rotated[:, 1][y_iris == idx],
             style, label=f"Iris {name}")

plt.xlabel("$z_1$")
plt.ylabel("$z_2$", rotation=0)
th1, th2 = tree_clf_pca.tree_.threshold[[0, 2]]
plt.plot([th1, th1], axes[2:], "k-", linewidth=2)
plt.plot([th2, th2], axes[2:], "k--", linewidth=2)
plt.text(th1 - 0.01, axes[2] + 0.05, "Depth=0",
         horizontalalignment="right", fontsize=15)
plt.text(th2 - 0.01, axes[2] + 0.05, "Depth=1",
         horizontalalignment="right", fontsize=13)
plt.axis(axes)
plt.legend(loc=(0.32, 0.67))
save_fig("pca_preprocessing_plot")

plt.show()
```



▼ Cây quyết định có phương sai cao

Chúng ta đã thấy rằng những thay đổi nhỏ trong tập dữ liệu (như xoay) có thể tạo ra một Cây quyết định rất khác. Bây giờ hãy chỉ ra rằng việc huấn luyện cùng một mô hình trên cùng một dữ liệu có thể tạo ra một mô hình rất khác nhau mỗi lần, vì thuật toán huấn luyện CART được Scikit-Learn sử dụng có tính ngẫu nhiên. Để chứng minh điều này, chúng ta sẽ đặt `random_state` thành một giá trị khác so với trước đó:

```
tree_clf_tweaked = DecisionTreeClassifier(max_depth=2, random_state=40)
tree_clf_tweaked.fit(X_iris, y_iris)

DecisionTreeClassifier(max_depth=2, random_state=40)
```

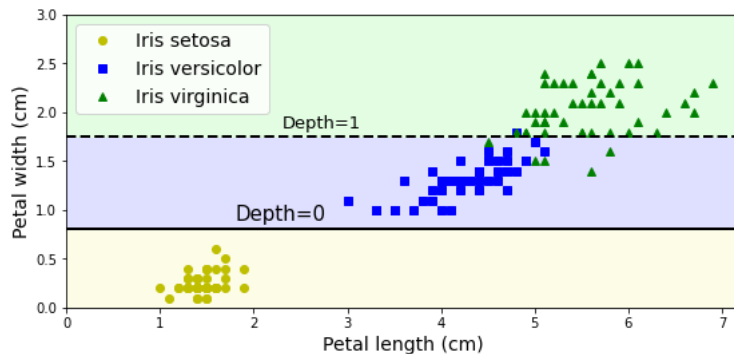
```
# extra code - this cell generates and saves Figure 6-9

plt.figure(figsize=(8, 4))
y_pred = tree_clf_tweaked.predict(X_iris_all).reshape(lengths.shape)
plt.contourf(lengths, widths, y_pred, alpha=0.3, cmap=custom_cmap)

for idx, (name, style) in enumerate(zip(iris.target_names, ("yo", "bs", "g^"))):
    plt.plot(X_iris[:, 0][y_iris == idx], X_iris[:, 1][y_iris == idx],
             style, label=f"Iris {name}")

th0, th1 = tree_clf_tweaked.tree_.threshold[[0, 2]]
plt.plot([0, 7.2], [th0, th0], "k-", linewidth=2)
plt.plot([0, 7.2], [th1, th1], "k--", linewidth=2)
plt.text(1.8, th0 + 0.05, "Depth=0", verticalalignment="bottom", fontsize=15)
plt.text(2.3, th1 + 0.05, "Depth=1", verticalalignment="bottom", fontsize=13)
plt.xlabel("Petal length (cm)")
plt.ylabel("Petal width (cm)")
plt.axis([0, 7.2, 0, 3])
plt.legend()
save_fig("decision_tree_high_variance_plot")

plt.show()
```



▼ Extra Material – Accessing the tree structure

A trained `DecisionTreeClassifier` has a `tree_` attribute that stores the tree's structure:

```
tree = tree_clf.tree_
tree

<sklearn.tree._tree.Tree at 0x7fbfa8b563b0>
```

You can get the total number of nodes in the tree:

```
tree.node_count

5
```

And other self-explanatory attributes are available:

```
tree.max_depth
```

```
2
```

```
tree.max_n_classes
```

```
3
```

```
tree.n_features
```

```
2
```

```
tree.n_outputs
```

```
1
```

```
tree.n_leaves
```

```
3
```

All the information about the nodes is stored in NumPy arrays. For example, the impurity of each node:

```
tree.impurity
```

```
array([0.66666667, 0.          , 0.5          , 0.16803841, 0.04253308])
```

The root node is at index 0. The left and right children nodes of node *i* are `tree.children_left[i]` and `tree.children_right[i]`. For example, the children of the root node are:

```
tree.children_left[0], tree.children_right[0]
```

```
(1, 2)
```

When the left and right nodes are equal, it means this is a leaf node (and the children node ids are arbitrary):

```
tree.children_left[3], tree.children_right[3]
```

```
(-1, -1)
```

So you can get the leaf node ids like this:

```
is_leaf = (tree.children_left == tree.children_right)  
np.arange(tree.node_count)[is_leaf]
```

```
array([1, 3, 4])
```

Non-leaf nodes are called *split nodes*. The feature they split is available via the `feature` array. Values for leaf nodes should be ignored:

```
tree.feature
```

```
array([ 0, -2,  1, -2, -2], dtype=int64)
```

And the corresponding thresholds are:

```
tree.threshold
```

```
array([ 2.44999999, -2.          ,  1.75          , -2.          , -2.          ])
```

And the number of instances per class that reached each node is available too:

```
tree.value
```

```
array([[50., 50., 50.],  
      [[50.,  0.,  0.]])
```

```
[[ 0., 50., 50.]],  
[[ 0., 49., 5.]],  
[[ 0., 1., 45.]])
```

```
tree.n_node_samples
```

```
array([150, 50, 100, 54, 46], dtype=int64)
```

```
np.all(tree.value.sum(axis=(1, 2)) == tree.n_node_samples)
```

```
True
```

Here's how you can compute the depth of each node:

```
def compute_depth(tree_clf):  
    tree = tree_clf.tree_  
    depth = np.zeros(tree.node_count)  
    stack = [(0, 0)]  
    while stack:  
        node, node_depth = stack.pop()  
        depth[node] = node_depth  
        if tree.children_left[node] != tree.children_right[node]:  
            stack.append((tree.children_left[node], node_depth + 1))  
            stack.append((tree.children_right[node], node_depth + 1))  
    return depth  
  
depth = compute_depth(tree_clf)  
depth  
  
array([0., 1., 1., 2., 2.])
```

Here's how to get the thresholds of all split nodes at depth 1:

```
tree_clf.tree_.feature[(depth == 1) & (~is_leaf)]
```

```
array([1], dtype=int64)
```

```
tree_clf.tree_.threshold[(depth == 1) & (~is_leaf)]
```

```
array([1.75])
```

✧ Lời giải bài tập

✧ 1. to 6.

1. The depth of a well-balanced binary tree containing m leaves is equal to $\log_2(m)$, rounded up. $\log_2$ is the binary log; $\log_2(m) = \log(m) / \log(2)$. A binary Decision Tree (one that makes only binary decisions, as is the case with all trees in Scikit-Learn) will end up more or less well balanced at the end of training, with one leaf per training instance if it is trained without restrictions. Thus, if the training set contains one million instances, the Decision Tree will have a depth of $\log_2(10^6) \approx 20$ (actually a bit more since the tree will generally not be perfectly well balanced).
2. A node's Gini impurity is generally lower than its parent's. This is due to the CART training algorithm's cost function, which splits each node in a way that minimizes the weighted sum of its children's Gini impurities. However, it is possible for a node to have a higher Gini impurity than its parent, as long as this increase is more than compensated for by a decrease in the other child's impurity. For example, consider a node containing four instances of class A and one of class B. Its Gini impurity is $1 - (1/5)^2 - (4/5)^2 = 0.32$. Now suppose the dataset is one-dimensional and the instances are lined up in the following order: A, B, A, A, A. You can verify that the algorithm will split this node after the second instance, producing one child node with instances A, B, and the other child node with instances A, A, A. The first child node's Gini impurity is $1 - (1/2)^2 - (1/2)^2 = 0.5$, which is higher than its parent's. This is compensated for by the fact that the other node is pure, so its overall weighted Gini impurity is $2/5 \times 0.5 + 3/5 \times 0 = 0.2$, which is lower than the parent's Gini impurity.

3. If a Decision Tree is overfitting the training set, it may be a good idea to decrease `max_depth`, since this will constrain the model, regularizing it.
4. Decision Trees don't care whether or not the training data is scaled or centered; that's one of the nice things about them. So if a Decision Tree underfits the training set, scaling the input features will just be a waste of time.
5. The computational complexity of training a Decision Tree is $O(n \times m \log_2(m))$. So if you multiply the training set size by 10, the training time will be multiplied by $K = (n \times 10 \times m \times \log_2(10 \times m)) / (n \times m \times \log_2(m)) = 10 \times \log_2(10 \times m) / \log_2(m)$. If $m = 10^6$, then $K \approx 11.7$, so you can expect the training time to be roughly 11.7 hours.
6. If the number of features doubles, then the training time will also roughly double.

7.

Exercise: train and fine-tune a Decision Tree for the moons dataset.

- a. Generate a moons dataset using `make_moons(n_samples=10000, noise=0.4)`.

Adding `random_state=42` to make this notebook's output constant:

```
from sklearn.datasets import make_moons

X_moons, y_moons = make_moons(n_samples=10000, noise=0.4, random_state=42)
```

- b. Split it into a training set and a test set using `train_test_split()`.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_moons, y_moons,
                                                    test_size=0.2,
                                                    random_state=42)
```

- c. Use grid search with cross-validation (with the help of the `GridSearchCV` class) to find good hyperparameter values for a `DecisionTreeClassifier`. Hint: try various values for `max_leaf_nodes`.

```
from sklearn.model_selection import GridSearchCV

params = {
    'max_leaf_nodes': list(range(2, 100)),
    'max_depth': list(range(1, 7)),
    'min_samples_split': [2, 3, 4]
}
grid_search_cv = GridSearchCV(DecisionTreeClassifier(random_state=42),
                              params,
                              cv=3)

grid_search_cv.fit(X_train, y_train)

GridSearchCV(cv=3, estimator=DecisionTreeClassifier(random_state=42),
             param_grid={'max_depth': [1, 2, 3, 4, 5, 6],
                         'max_leaf_nodes': [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,
                                             13, 14, 15, 16, 17, 18, 19, 20, 21,
                                             22, 23, 24, 25, 26, 27, 28, 29, 30,
                                             31, ...],
                         'min_samples_split': [2, 3, 4]})
```

```
grid_search_cv.best_estimator_
```

```
DecisionTreeClassifier(max_depth=6, max_leaf_nodes=17, random_state=42)
```

- d. Train it on the full training set using these hyperparameters, and measure your model's performance on the test set. You should get roughly 85% to 87% accuracy.

By default, `GridSearchCV` trains the best model found on the whole training set (you can change this by setting `refit=False`), so we don't need to do it again. We can simply evaluate the model's accuracy:

```
from sklearn.metrics import accuracy_score

y_pred = grid_search_cv.predict(X_test)
accuracy_score(y_test, y_pred)

0.8595
```

8.

Exercise: Grow a forest.

a. Continuing the previous exercise, generate 1,000 subsets of the training set, each containing 100 instances selected randomly. Hint: you can use Scikit-Learn's `ShuffleSplit` class for this.

```
from sklearn.model_selection import ShuffleSplit

n_trees = 1000
n_instances = 100

mini_sets = []

rs = ShuffleSplit(n_splits=n_trees, test_size=len(X_train) - n_instances,
                  random_state=42)

for mini_train_index, mini_test_index in rs.split(X_train):
    X_mini_train = X_train[mini_train_index]
    y_mini_train = y_train[mini_train_index]
    mini_sets.append((X_mini_train, y_mini_train))
```

b. Train one Decision Tree on each subset, using the best hyperparameter values found above. Evaluate these 1,000 Decision Trees on the test set. Since they were trained on smaller sets, these Decision Trees will likely perform worse than the first Decision Tree, achieving only about 80% accuracy.

```
from sklearn.base import clone

forest = [clone(grid_search_cv.best_estimator_) for _ in range(n_trees)]

accuracy_scores = []

for tree, (X_mini_train, y_mini_train) in zip(forest, mini_sets):
    tree.fit(X_mini_train, y_mini_train)

    y_pred = tree.predict(X_test)
    accuracy_scores.append(accuracy_score(y_test, y_pred))

np.mean(accuracy_scores)

0.805671
```

c. Now comes the magic. For each test set instance, generate the predictions of the 1,000 Decision Trees, and keep only the most frequent prediction (you can use SciPy's `mode()` function for this). This gives you *majority-vote predictions* over the test set.

```
Y_pred = np.empty([n_trees, len(X_test)], dtype=np.uint8)

for tree_index, tree in enumerate(forest):
    Y_pred[tree_index] = tree.predict(X_test)
```

```
from scipy.stats import mode

y_pred_majority_votes, n_votes = mode(Y_pred, axis=0)
```

d. Evaluate these predictions on the test set: you should obtain a slightly higher accuracy than your first model (about 0.5 to 1.5% higher). Congratulations, you have trained a Random Forest classifier!

```
accuracy_score(y_test, y_pred_majority_votes.reshape([-1]))
```

